

Arquitectura Binario Único — Gestor-Horarios

Documento: Arquitectura destino del cambio `arquitectura-binario-unico` **Fecha:** 5 de julio de 2026 **Autor:** Luis Rojas - Teach-lead **Estado:** Propuesta técnica

Índice

- 1. [Resumen Ejecutivo](#)
- 2. [Arquitectura Actual](#)
- 3. [Problemas Detectados](#)
- 4. [Arquitectura Destino](#)
- 5. [Alternativa Elegida: QProcess](#)
- 6. [Diagrama de Componentes](#)
- 7. [Flujo de Arranque](#)
- 8. [Mecanismo de Crash Recovery](#)
- 9. [Cambios Archivo por Archivo](#)
- 10. [Especificación CMake](#)
- 11. [Compilación Condicional](#)
- 12. [Plan de Implementación \(5 Fases\)](#)
- 13. [Evaluación de Riesgos](#)
- 14. [Comandos de Compilación](#)
- 15. [Impacto en Issues del Backlog](#)
- 16. [Archivos de Referencia](#)

1. Resumen Ejecutivo

El proyecto Gestor-Horarios genera actualmente **2 ejecutables independientes** que se comunican por IPC (QLocalSocket). Esto obliga a todo desarrollador a tener Docker (con OR-Tools y Google Test) para compilar, incluso si solo trabajan en la interfaz gráfica.

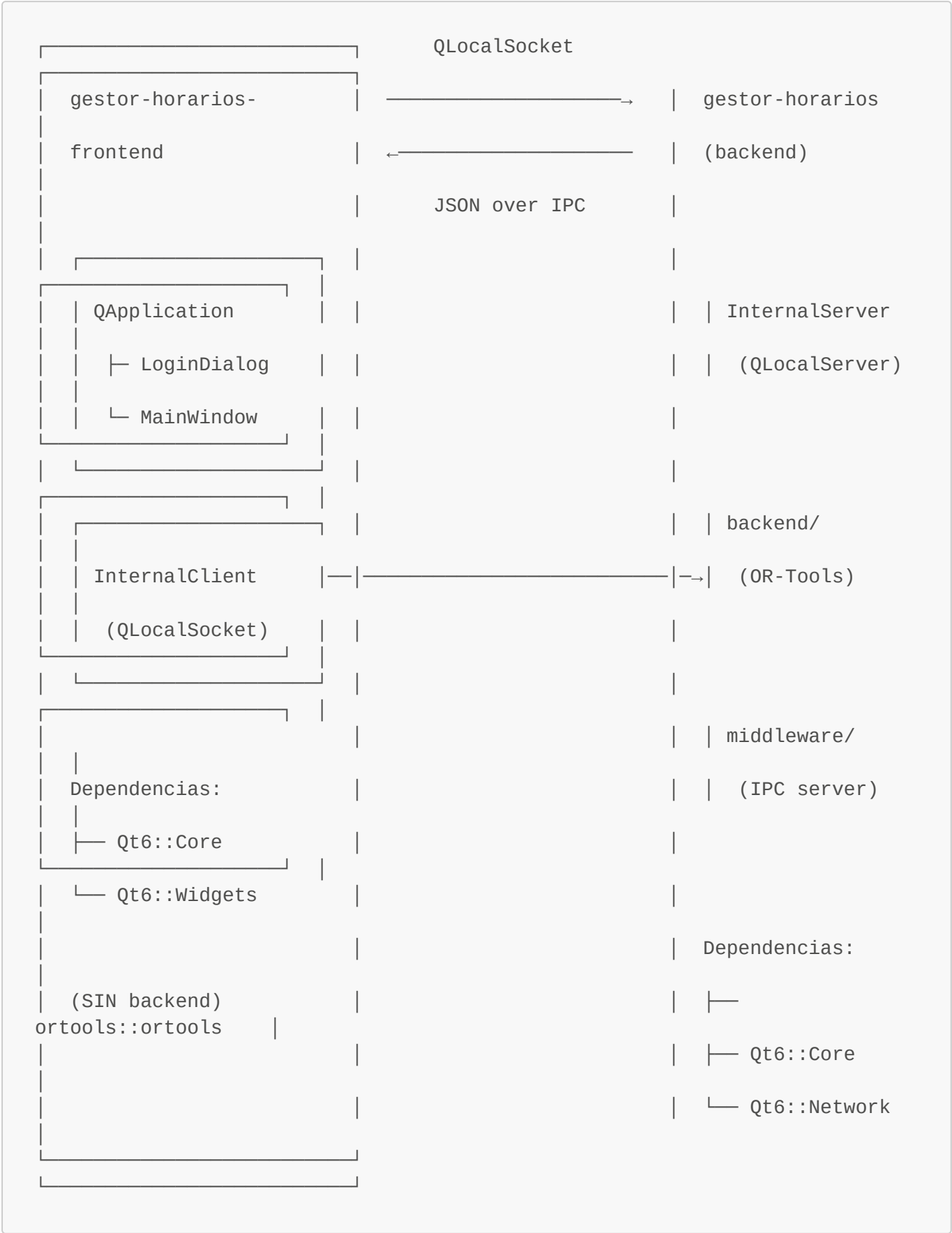
Este cambio propone consolidar el sistema en un ÚNICO binario distribuible, manteniendo el aislamiento de procesos (backend y frontend en procesos separados) y habilitando la compilación condicional para que el equipo de frontend pueda compilar sin Docker.

¿Qué cambia?

Aspecto	Hoy	Después
Binarios generados	2	1
Entry point	Cada ejecutable tiene su <code>main()</code>	Un solo <code>main()</code> con dual-mode
Aislamiento procesos	Manual (el usuario lanza ambos)	Auto (QProcess lanza backend)
Frontend standalone	Sí (ya funciona)	Sí + flag <code>FRONTEND_STANDALONE</code>
OR-Tools	Requerido siempre	Opcional (<code>BUILD_BACKEND</code>)

Aspecto	Hoy	Después
Middleware IPC	Solo entre procesos	Sin cambios (se reutiliza)

2. Arquitectura Actual



Módulos del proyecto

Módulo	Ruta	Tipo	Propósito
app/	src/app/	Ejecutable	Punto de entrada (hoy placeholder)
frontend/	src/frontend/	Ejecutable (standalone)	UI Qt6 Widgets
backend/	src/backend/	Librería estática	OR-Tools + datos
middleware/	src/middleware/	Librería estática	IPC QLocalSocket
common/	src/common/	Header-only	Tipos compartidos

IPC actual

- **Mecanismo:** QLocalServer / QLocalSocket (named pipes en Unix, pipes en Windows)
- **Formato:** JSON (QJsonDocument / QJsonObject)
- **Mensajes implementados:** Solo `health_check` (envía `{"op": "health_check"}`, recibe `{"status": "ok"}`)
- **Código:** `src/middleware/` — servidor en `internalserver.cpp`, cliente en `internalclient.cpp`

3. Problemas Detectados

P1: OR-Tools requerido incondicionalmente

```
# Root CMakeLists.txt — LÍNEA PROBLEMÁTICA
find_package(ortools CONFIG REQUIRED)
```

Esto forza a **cualquier persona** que quiera compilar el proyecto a tener OR-Tools instalado. OR-Tools solo vive dentro del Docker. Sin Docker → no se puede compilar ni siquiera el frontend.

P2: BUILD_BACKEND no tiene `option()`

La variable `BUILD_BACKEND` existe en `src/CMakeLists.txt` pero nunca se declara con `option()`. No aparece en `cmake -L` y nadie sabe que existe.

P3: Frontend tiene `project()` anidado

```
# src/frontend/CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
project(gestor-horarios-frontend VERSION 1.0.0 LANGUAGES CXX)
```

Tener un `project()` dentro de otro `project()` es un anti-patrón en CMake. El hijo no hereda bien las variables del padre y crea un scope separado.

P4: OR-Tools linkeado como PUBLIC

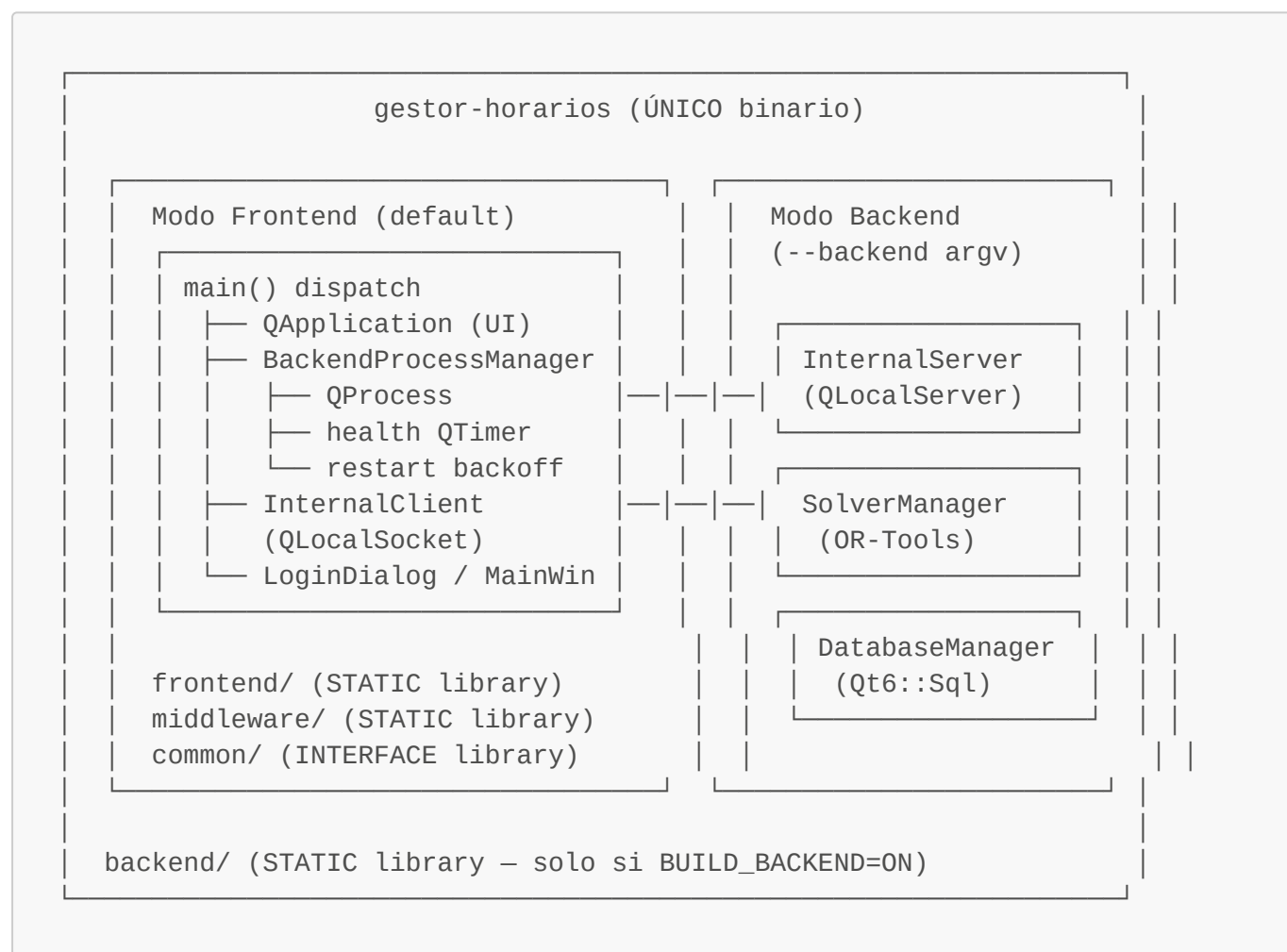
```
# src/backend/CMakeLists.txt
target_link_libraries(backend PUBLIC ortools::ortools)
```

Cualquier target que linkee **backend** se ve forzado a tener OR-Tools en su ruta de include, incluso si no lo usa directamente.

P5: **common/** está vacío

El módulo **common/** existe estructuralmente como **INTERFACE** library pero no tiene ningún archivo. No hay códigos de error, constantes ni tipos compartidos.

4. Arquitectura Destino



Principios de diseño

- 1. Un solo binario, dos modos:** El ejecutable decide su rol según **argv**
- 2. Aislamiento por procesos:** Frontend y backend son procesos separados (QProcess)
- 3. IPC reutilizado:** El middleware QLocalSocket no se refactoriza
- 4. Compilación condicional:** **BUILD_BACKEND** controla dependencias pesadas
- 5. Retrocompatibilidad:** **FRONTEND_STANDALONE** preserva el flujo Qt Creator

5. Alternativa Elegida: QProcess

Se evaluaron 4 alternativas técnicas. La elegida es la **Alternativa A: QProcess**.

Alternativa	Técnica	Aislamiento	Esfuerzo	Veredicto
A ★	QProcess — auto-lanzamiento	✓ Sí (procesos separados)	6-8h	Elegida
B	Thread isolation (QThread)	✗ Crash mata todo	4-5h	Descartada
C	fork+exec self	✓ Sí	~6h	Descartada (QProcess > fork)
D	Signal isolation (sigaction)	✗ Undefined behavior	N/A	Inviable

¿Por qué QProcess?

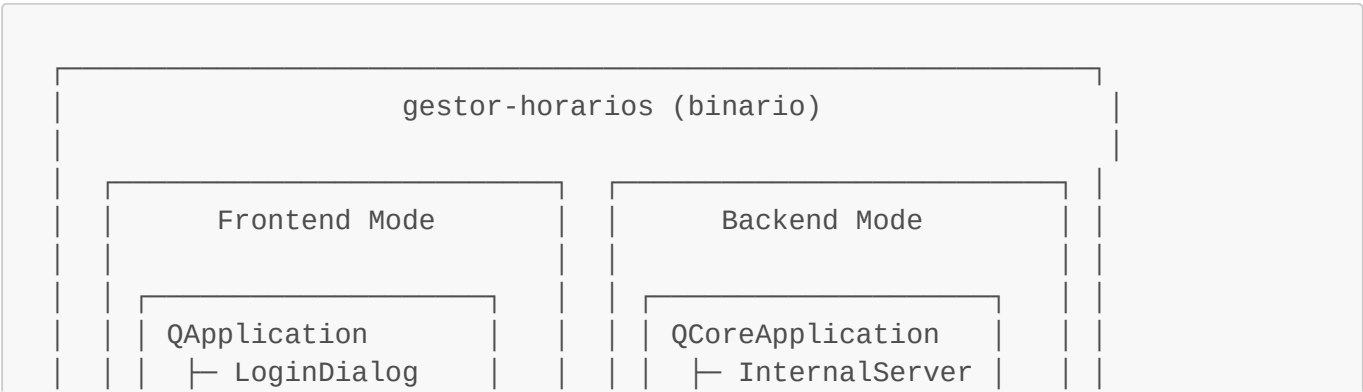
Razón	Explicación
Aislamiento real	Si la UI crashea (segfault en Qt event loop), el backend child sigue corriendo
Cross-platform	QProcess funciona en Linux, Windows (CreateProcess), macOS
IPC reutilizado	El middleware QLocalSocket/JSON se mantiene exactamente igual
Mínimo riesgo	No se refactoriza el middleware existente, solo se agrega código nuevo
Qt nativo	El equipo conoce Qt, QProcess es parte de Qt6::Core

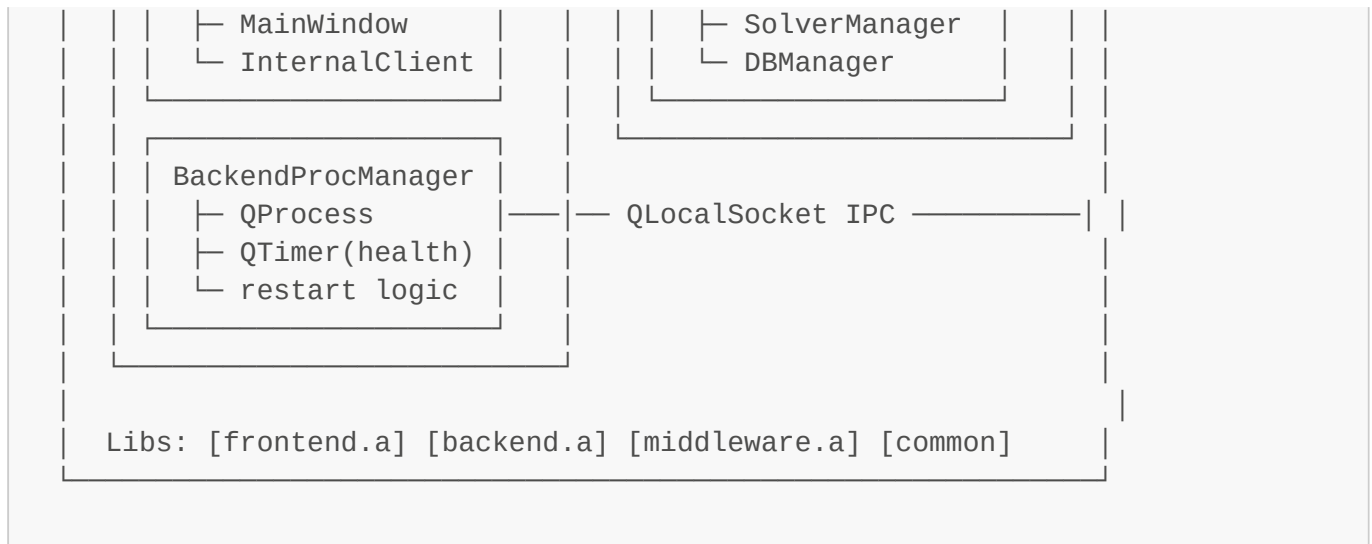
¿Por qué se descartaron las otras?

- **Thread isolation:** Cualquier segfault en la UI mata el proceso completo. No cumple el requisito.
- **fork+exec:** QProcess ya encapsula fork+exec UNIX y CreateProcess Windows. No reinventar.
- **Signal isolation:** Comportamiento indefinido post-SIGSEGV según el estándar C++. Inviable.

6. Diagrama de Componentes

Arquitectura estática





Dependencias entre módulos

```

gestor-horarios (ejecutable)
├── common/      → header-only (INTERFACE)
├── frontend/    → Qt6::Core, Qt6::Widgets (STATIC)
├── middleware/  → Qt6::Core, Qt6::Network (STATIC)
├── backend/     → ortools::ortools, Qt6::Core (STATIC, solo
BUILD_BACKEND=ON)
  
```

7. Flujo de Arranque

Modo Frontend (default)

Usuario: `./gestor-horarios`

```

main()
├── argv sin "--backend" → run_frontend_app()
├── [1] BackendProcessManager::start()
│   ├── QProcess("gestor-horarios --backend")
│   ├── wait 1.5s (esperar a que abra el socket)
│   └── health_check → recibir {"status":"ok"} → emit backendReady
├── [2] LoginDialog.exec()
│   ├── usuario ingresa credenciales
│   └── aceptado → MainWindow.show()
├── [3] InternalClient conecta al socket
│   └── Envía/recibe operaciones IPC según la UI
└── [4] QTimer cada 5s → health-check automático
    └── Si falla → detectar crash → reiniciar backend
  
```

Modo Backend (--backend)

Usuario: `./gestor-horarios --backend`

```
main()
├─ argv contiene "--backend" → run_backend_app()
├─ [1] QApplication (sin UI, sin QApplication)
├─ [2] InternalServer::start()
│   ├── QLocalServer::removeServer("GestorHorarios_Middleware")
│   ├── m_server->listen("GestorHorarios_Middleware")
│   └─ connect(newConnection → onNewConnection)
├─ [3] SolverManager::ready() (TODO: implementar)
├─ [4] DatabaseManager::initialize() (TODO: implementar)
└─ [5] QApplication::exec() – event loop
    └─ Espera conexiones IPC, procesa operaciones
```

Shutdown

Usuario: `Ctrl+C` o cierra ventana

```
BackendProcessManager::stop()
├─ [1] Enviar OP_SHUTDOWN vía IPC al backend
├─ [2] QProcess::terminate() (SIGTERM en Unix)
├─ [3] waitForFinished(5000ms)
├─ [4] Si no responde → QProcess::kill() (SIGKILL)
└─ [5] app.exit()
```

8. Mecanismo de Crash Recovery

Backend crash → Frontend reinicia

```
BackendProcessManager
├─ health-check OK (cada 5s)
├─ QTimer cada 5s → InternalClient::sendHealthCheck()
├─ Si health-check RESPONDE → ok, seguir
├─ Si health-check NO RESPONDE (timeout 10s)
│   ├── QProcess::CrashExit signal?
│   │   ├── Sí → crash detectado
│   │   └─ No → QProcess::kill() (proceso colgado)
```

```
├─ emit backendCrashed()
└─ restart con backoff exponencial:
    ├─ Intento 1: esperar 2s
    ├─ Intento 2: esperar 4s
    ├─ Intento 3: esperar 6s
    └─ Si 3 intentos fallan → emit backendCrashed(final)
└─ UI muestra mensaje: "Reconectando con el servidor..."
```

Frontend crash → Backend sigue vivo

```

Procesos separados → el backend ni se entera
├─ BackendProcessManager (en proceso frontend) muere con la UI
├─ El backend child process sigue ejecutándose
├─ QLocalSocket se cierra (el peer desapareció)
└─ Backend detecta disconnected → cleanup resources

```

Health-check



Configuración del BackendProcessManager

Parámetro	Valor	Descripción
HEALTH_CHECK_INTERVAL_MS	5000	Cada 5s se envía health-check
INITIAL_WAIT_MS	1500	Espera inicial tras lanzar backend
BACKOFF_BASE_MS	2000	Backoff base para reintentos
MAX_RESTART_ATTEMPTS	3	Máximo de reintentos antes de rendirse

9. Cambios Archivo por Archivo

Archivos NUEVOS (8)

Archivo	Propósito	Líneas estimadas
<code>src/app/main.cpp</code>	Entry point dual-mode: <code>parsea --backend</code>	~30
<code>src/app/include/app/backend_process_manager.hpp</code>	Header BackendProcessManager	~40
<code>src/app/src/backend_process_manager.cpp</code>	QProcess lifecycle + health-check + backoff	~120
<code>src/app/include/app/frontend_application.hpp</code>	Declara <code>run_frontend_app()</code>	~10
<code>src/app/src/frontend_application.cpp</code>	Orquesta UI + BPM + Login	~40
<code>src/app/include/app/backend_application.hpp</code>	Declara <code>run_backend_app()</code>	~10
<code>src/app/src/backend_application.cpp</code>	Modo servidor backend	~30
<code>src/common/include/common/errors.hpp</code>	Códigos de error del sistema	~30
<code>src/common/include/common/version.hpp</code>	Constantes de versión	~10

Archivos MODIFICADOS (5)

Archivo	Cambio
<code>CMakeLists.txt</code> (root)	<code>option(BUILD_BACKEND)</code> , <code>find_package(ortools)</code> condicional, <code>option(BUILD_TESTING)</code>
<code>src/CMakeLists.txt</code>	<code>middleware</code> se compila siempre (no solo con <code>BUILD_BACKEND</code>)
<code>src/app/CMakeLists.txt</code>	Linkea <code>frontend</code> , <code>backend</code> (condicional), <code>middleware</code> . Nuevas fuentes.
<code>src/frontend/CMakeLists.txt</code>	De <code>project()</code> → <code>add_library(frontend</code> <code>STATIC)</code> . Flag <code>FRONTEND_STANDALONE</code> .
<code>src/middleware/include/middleware/messages.h</code>	Agrega <code>OP_SHUTDOWN</code> , <code>OP_READY</code> , códigos de respuesta
<code>src/middleware/include/middleware/internalclient.h</code>	Agrega <code>sendRequest()</code> genérico

Archivo	Cambio
<code>src/middleware/src/client/internalclient.cpp</code>	Implementa <code>sendRequest()</code> , refactoriza <code>sendHealthCheck()</code>

Archivos ELIMINADOS (2)

Archivo	Razón
<code>src/app/main_placeholder.cpp</code>	Reemplazado por <code>main.cpp</code> real
<code>src/frontend/src/main.cpp</code>	Contenido movido a <code>frontend_application.cpp</code> (se mantiene para standalone)

Archivos SIN CAMBIOS (la mayoría)

Módulo	Archivos	Estado
<code>backend/</code>	<code>CMakeLists.txt</code> , <code>include/backend/data/*.hpp</code> , <code>src/data/*.cpp</code>	✓ Sin cambios
<code>frontend/</code>	<code>src/logindialog.*</code> , <code>src/views/*</code>	✓ Sin cambios
<code>middleware/</code>	<code>internalserver.*</code>	✓ Mínimos cambios (solo routing)
<code>common/</code>	(antes vacío)	✓ Se puebla
<code>test/</code>	tests existentes	✓ Sin cambios estructurales
<code>docker/</code>	<code>Dockerfile.dev</code> , <code>docker-compose.yml</code>	✓ Sin cambios
<code>cmake/</code>	<code>CompilerWarnings.cmake</code>	✓ Sin cambios

10. Especificación CMake

Root `CMakeLists.txt` (post-cambio)

```
cmake_minimum_required(VERSION 3.24)
project(Gestor-Horarios
  VERSION 0.1.0
  DESCRIPTION "Sistema de gestión de horarios con optimización (OR-Tools)"
  LANGUAGES CXX
)

# — Opciones de compilación —
option(BUILD_BACKEND "Build backend, middleware, and solver (requires OR-Tools)" ON)
option(BUILD_TESTING "Build tests" ON)
```

```
# — Estándar C++ —
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# — Dependencias externas —
find_package(Qt6 REQUIRED COMPONENTS Core Sql Widgets Network Test)

# OR-Tools: solo si BUILD_BACKEND=ON
if(BUILD_BACKEND)
    find_package(ortools CONFIG REQUIRED)
    message(STATUS "OR-Tools found — backend and solver enabled")
else()
    message(STATUS "BUILD_BACKEND=OFF — backend and OR-Tools disabled")
endif()

# — Warnings —
include(cmake/CompilerWarnings.cmake)

# — Módulos —
add_subdirectory(src)

# — Tests —
if(BUILD_TESTING)
    enable_testing()
    add_subdirectory(test)
endif()
```

src/CMakeLists.txt (post-cambio)

```
add_subdirectory(common)
add_subdirectory(frontend)
add_subdirectory(middleware) # middleware siempre necesario para IPC

if(BUILD_BACKEND)
    add_subdirectory(backend)
endif()

add_subdirectory(app)
```

src/app/CMakeLists.txt (post-cambio)

```
add_executable(gestor-horarios)

target_sources(gestor-horarios PRIVATE
    main.cpp
    src/backend_process_manager.cpp
    src/frontend_application.cpp
    src/backend_application.cpp
)
```

```

target_link_libraries(gestor-horarios PRIVATE
    common
    frontend
)

if(TARGET backend)
    target_link_libraries(gestor-horarios PRIVATE backend)
endif()
if(TARGET middleware)
    target_link_libraries(gestor-horarios PRIVATE middleware)
endif()

set_project_warnings(gestor-horarios)

```

src/frontend/CMakeLists.txt (post-cambio)

```

# Opción para desarrollo frontend standalone (Qt Creator)
option(FRONTEND_STANDALONE "Build as standalone executable for Qt Creator"
OFF)

set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTORCC ON)

set(FRONTEND_SOURCES
    src/frontend_application.cpp
    src/logindialog.cpp
)

set(FRONTEND_HEADERS
    include/frontend/frontend_application.hpp
    src/logindialog.h
)

if(FRONTEND_STANDALONE)
    # === MODO Qt CREATOR ===
    # Ejecutable independiente con main() legacy
    qt6_add_executable(gestor-horarios-frontend
        ${FRONTEND_SOURCES}
        ${FRONTEND_HEADERS}
        src/main.cpp
    )
    target_link_libraries(gestor-horarios-frontend PRIVATE
        Qt6::Core
        Qt6::Widgets
    )
    target_compile_definitions(gestor-horarios-frontend PRIVATE
        PROJECT_VERSION="${PROJECT_VERSION}"
    )
else()

```

```
# === MODO INTEGRADO ===
# Librería estática para linkear en el binario único
add_library(frontend STATIC)
target_sources(frontend PRIVATE
    ${FRONTEND_SOURCES}
    ${FRONTEND_HEADERS}
)
target_include_directories(frontend PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)
target_include_directories(frontend PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/src
)
target_link_libraries(frontend PUBLIC
    Qt6::Core
    Qt6::Widgets
)
target_compile_definitions(frontend PUBLIC
    PROJECT_VERSION="${PROJECT_VERSION}"
)
endif()
```

11. Compilación Condicional

Flags disponibles

Flag	Valores	Default	Descripción
BUILD_BACKEND	ON / OFF	ON	Compila backend + solver (requiere OR-Tools)
BUILD_TESTING	ON / OFF	ON	Compila y ejecuta tests
FRONTEND_STANDALONE	ON / OFF	OFF	Compila frontend como ejecutable legacy (Qt Creator)

Matriz de configuraciones

Configuración	Comando	¿Requiere Docker?	¿Qué compila?	Para quién
Full system	cmake -S . -B build	✓ Sí	Binario único completo + tests	CI / Release
Frontend-only	cmake -S . -B build - DBUILD_BACKEND=OFF	✗ No	Binario único (solo modo frontend)	Desarrolladores sin Docker

Configuración	Comando	¿Requiere Docker?	¿Qué compila?	Para quién
Qt Creator	Abrir <code>src/frontend/CMakeLists.txt</code> + <code>-DFRONTEND_STANDALONE=ON</code>	✗ No	Ejecutable frontend legacy	Dani / Paola
Sin tests	<code>cmake -S . -B build -</code> <code>DBUILD_TESTING=OFF</code>	✓✗	Sin tests (compilación más rápida)	Desarrollo rápido

Dependencias por flag

Flag activo	Qt6	OR-Tools	Google Test
<code>BUILD_BACKEND=ON + BUILD_TESTING=ON</code>	✓	✓	✓
<code>BUILD_BACKEND=ON + BUILD_TESTING=OFF</code>	✓	✓	✗
<code>BUILD_BACKEND=OFF + BUILD_TESTING=ON</code>	✓	✗	✗
<code>BUILD_BACKEND=OFF + BUILD_TESTING=OFF</code>	✓	✗	✗
<code>FRONTEND_STANDALONE=ON</code>	✓ (solo Core+Widgets)	✗	✗

12. Plan de Implementación (5 Fases)

Fase 1 — Foundation (REFAC-1) ~2h

Hacer OR-Tools condicional + opciones CMake

#	Tarea	Archivo
1	Agregar <code>option(BUILD_BACKEND ON)</code>	<code>CMakeLists.txt</code> root
2	Mover <code>find_package(ortools)</code> dentro de <code>if(BUILD_BACKEND)</code>	<code>CMakeLists.txt</code> root
3	Agregar <code>option(BUILD_TESTING ON)</code> + tests condicionales	<code>CMakeLists.txt</code> root
4	Sacar <code>middleware</code> del <code>if(BUILD_BACKEND)</code> en <code>src/</code>	<code>src/CMakeLists.txt</code>

Fase 2a — Frontend library (REFAC-2) ~3h

Convertir frontend de ejecutable a librería estática

#	Tarea	Archivo
1	Eliminar <code>project()</code> y <code>qt6_add_executable()</code>	<code>src/frontend/CMakeLists.txt</code>
2	Agregar <code>option(FRONTEND_STANDALONE)</code> con modo dual	<code>src/frontend/CMakeLists.txt</code>
3	Crear <code>frontend_application.hpp</code>	<code>src/frontend/include/frontend/</code>

#	Tarea	Archivo
4	Crear <code>frontend_application.cpp</code> con lógica migrada	<code>src/frontend/src/</code>
5	Mantener <code>main.cpp</code> legacy solo para standalone	<code>src/frontend/src/main.cpp</code>

Fase 2b — main dual-mode (REFAC-3) ~4h

Crear entry point único con parseo de argv

#	Tarea	Archivo
1	Crear <code>main.cpp</code> con parseo de argv	<code>src/app/main.cpp</code>
2	Crear <code>frontend_application.hpp/.cpp</code> (orquestador)	<code>src/app/src/</code>
3	Crear <code>backend_application.hpp/.cpp</code> (servidor)	<code>src/app/src/</code>
4	Actualizar <code>src/app/CMakeLists.txt</code>	<code>src/app/CMakeLists.txt</code>
5	Eliminar <code>main_placeholder.cpp</code>	<code>src/app/</code>

Fase 3 — BackendProcessManager (REFAC-4) ~5h

Clase QProcess con health-check + crash recovery

#	Tarea	Archivo
1	Crear header con API pública	<code>src/app/include/app/backend_process_manager.hpp</code>
2	Implementar <code>start()</code> → QProcess	<code>src/app/src/backend_process_manager.cpp</code>
3	Implementar health-check periódico (QTimer 5s)	<code>src/app/src/backend_process_manager.cpp</code>
4	Implementar crash detection + backoff (2s/4s/6s)	<code>src/app/src/backend_process_manager.cpp</code>
5	Implementar <code>stop()</code> graceful + force kill	<code>src/app/src/backend_process_manager.cpp</code>
6	Integrar BPM en <code>frontend_application.cpp</code>	<code>src/app/src/frontend_application.cpp</code>

Fase 4 — Middleware expansion (REFAC-5) ~3h

Extender protocolo IPC con nuevas operaciones

#	Tarea	Archivo
1	Agregar <code>OP_SHUTDOWN</code> , <code>OP_READY</code> , códigos	<code>src/middleware/include/middleware/messages.h</code>

#	Tarea	Archivo
	respuesta	
2	Agregar <code>sendRequest()</code> genérico en <code>InternalClient</code>	<code>src/middleware/src/client/internalclient.cpp</code>
3	Agregar <code>responseReceived</code> signal	<code>src/middleware/include/middleware/internalclient.h</code>
4	Manejar <code>OP_SHUTDOWN</code> , <code>OP_READY</code> en <code>InternalServer</code>	<code>src/middleware/src/server/internalserver.cpp</code>

Fase 5 — Tests (REFAC-6) ~4h

Tests de integración del launcher y build condicional

#	Tarea	Archivo
1	Test: <code>BUILD_BACKEND=OFF</code> compila sin OR-Tools	<code>test/</code>
2	Test: lanzar <code>--backend</code> , health-check respuesta "ok"	<code>test/</code>
3	Test: BPM detecta crash e intenta restart	<code>test/</code>
4	Verificar tests existentes siguen pasando	<code>ctest</code>

Fase 6 — Documentación (REFAC-7) ~3h

Documentar arquitectura + actualizar guías

#	Tarea	Archivo
1	Crear <code>docs/arquitectura-binario-unico.md</code> (este documento)	<code>docs/</code>
2	Generar PDF	<code>docs/</code>
3	Actualizar <code>README.md</code> con flags de compilación	raíz
4	Revisar <code>docs/guias/</code>	<code>docs/guias/</code>
5	Revisar <code>.github/workflows/</code>	<code>.github/</code>

13. Evaluación de Riesgos

#	Riesgo	Prob.	Impacto	Mitigación
R1	Qt Creator del equipo frontend deja de funcionar	Alta	Alto	Mantener <code>FRONTEND_STANDALONE=ON</code> para desarrollo standalone. Probar en Qt Creator antes de mergear.
R2	Regresión en middleware IPC	Media	Alto	Tests existentes (<code>test_health_check.cpp</code>) deben seguir pasando. Los tests de health-check se ejecutan en CI.

#	Riesgo	Prob.	Impacto	Mitigación
R3	QProcess no detecta correctamente crash	Media	Medio	Usar <code>QProcess::waitForFinished()</code> con timeout + health-check periódico como doble verificación.
R4	BUILD_BACKEND=OFF falla en CI	Baja	Alto	Agregar workflow de CI que compile con <code>-DBUILD_BACKEND=OFF</code> .
R5	Windows: ruta del ejecutable en QProcess	Baja	Bajo	<code>QCoreApplication::applicationFilePath()</code> funciona cross-platform.
R6	Backend lento en arrancar	Media	Medio	<code>INITIAL_WAIT_MS</code> configurable (1500ms). Reintento de health-check si timeout.
R7	Dos instancias del backend (race condition)	Baja	Alto	<code>QLocalServer::removeServer()</code> en <code>InternalServer::start()</code> previene esto.

Plan de rollback

Paso	Acción
1	<code>git tag pre-unified-binary HEAD</code> antes de empezar
2	Si Fase 1 falla → <code>git checkout pre-unified-binary</code>
3	Si Fase 2 falla → <code>git revert <commit-fase2></code>
4	Si Fase 3 falla → no mergeear sin test verde
5	Rollback total → <code>git revert --no-commit HEAD~N..HEAD</code> + commit único

14. Comandos de Compilación

Prerrequisitos

- CMake 3.24+
- Ninja (opcional, recomendado)
- Qt6 (Core, Sql, Widgets, Network, Test)
- OR-Tools 9.15+ (solo para BUILD_BACKEND=ON)
- Compilador C++17 (GCC 9+, Clang 10+, MSVC 2019+)

Build completo (con OR-Tools)

```
# Configurar
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug

# Compilar
cmake --build build
```

```
# Ejecutar tests
ctest --test-dir build --output-on-failure

# Ejecutar binario único
./build/src/app/gestor-horarios
```

Build frontend-only (sin OR-Tools, sin Docker)

```
# Configurar (NO requiere OR-Tools ni Docker)
cmake -S . -B build -G Ninja -DBUILD_BACKEND=OFF -DBUILD_TESTING=OFF

# Compilar
cmake --build build

# Ejecutar (solo modo frontend, sin backend real)
./build/src/app/gestor-horarios
```

Build frontend standalone (Qt Creator)

```
# Desde la terminal:
cmake -S src/frontend -B build-frontend -DFRONTEND_STANDALONE=ON
cmake --build build-frontend
./build-frontend/gestor-horarios-frontend

# O desde Qt Creator:
# 1. File → Open File or Project → src/frontend/CMakeLists.txt
# 2. En CMake configuration, agregar: -DFRONTEND_STANDALONE=ON
# 3. Build & Run
```

Ejecución en modo backend

```
# Iniciar solo el servidor backend (headless)
./build/src/app/gestor-horarios --backend

# Ver versión
./build/src/app/gestor-horarios --version

# Ver ayuda
./build/src/app/gestor-horarios --help
```

Docker (sin cambios)

```
# El Docker actual sigue funcionando exactamente igual
docker compose -f docker/docker-compose.yml up --build
```

CI / Verificación

```
# Verificar ambas configuraciones de build
cmake -S . -B build-full -G Ninja -DBUILD_BACKEND=ON && cmake --build
build-full
cmake -S . -B build-lite -G Ninja -DBUILD_BACKEND=OFF && cmake --build
build-lite

# Verificar frontend standalone
cmake -S src/frontend -B build-fe -DFRONTEND_STANDALONE=ON && cmake --build
build-fe
```

15. Impacto en Issues del Backlog

Sprint 2 (completado, 22-28 Jun)

Issue	Título	Impacto
S2-I1	OR-Tools + estructuras de datos	✓ Sin cambios
S2-I2	Endpoints CRUD middleware	✓ Sin cambios
S2-I3	Maqueta login/dashboard	✓ Sin cambios (solo cambia cómo se compila)
S2-I4	Setup middleware + health-check	✓ Sin cambios (se expande en Fase 4)
S2-I5	SQLite schema	✓ Sin cambios
S2-I6	Scaffold Qt + capacitación	⚠ CMake de frontend cambia: de ejecutable a librería. Mitigado por FRONTEND_STANDALONE .

Sprint 3 (en curso, 29 Jun-5 Jul)

Issue	Título	Impacto
S3-I1	Modelo de datos + diagrama ER	✓ Sin cambios
S3-I2	CRUD materias	✓ Sin cambios
S3-I3	Formulario profesores	✓ Sin cambios

Issue	Título	Impacto
S3-I4	Enrutamiento middleware	⚠ Afectado indirectamente: routing por op se expande en Fase 4
S3-I5	CRUD aulas	✅ Sin cambios
S3-I6	Prototipos dashboard + navegación	✅ Sin cambios

Sprint 4 (6-12 Jul)

Issue	Título	Impacto
S4-I1	CP-SAT solver	✅ Bloqueante resuelto : BUILD_BACKEND condicional
S4-I2	CRUD horarios	✅ Sin cambios
S4-I3	Formularios docente/sección	✅ Sin cambios
S4-I4	Tests unitarios modelo	✅ Sin cambios
S4-I5	Servicio consultas	✅ Sin cambios
S4-I6	Formularios Qt docente/sección	✅ Sin cambios

Nuevas tareas en el backlog

ID	Tarea	Fase	Esfuerzo	Asignado sugerido
REFAC-1	CMake condicional (OR-Tools opcional)	1	~2h	Luis
REFAC-2	Frontend como librería estática	2a	~3h	Dani / Paola
REFAC-3	main.cpp dual-mode + parseo argv	2b	~4h	Luis
REFAC-4	BackendProcessManager (QProcess)	3	~5h	Manuel
REFAC-5	Middleware expansion	4	~3h	Manuel
REFAC-6	Tests de integración	5	~4h	Manuel / Luis
REFAC-7	Documentación técnica	6	~3h	Luis

Total esfuerzo: ~24 horas

16. Archivos de Referencia

Archivo	Propósito
<code>CMakeLists.txt</code> (root)	Punto de entrada del build system
<code>src/CMakeLists.txt</code>	Orquestador de submódulos
<code>src/app/CMakeLists.txt</code>	Build del binario único

Archivo	Propósito
<code>src/frontend/CMakeLists.txt</code>	Build del frontend (library + standalone)
<code>src/app/main.cpp</code>	Entry point dual-mode
<code>src/app/src/backend_process_manager.cpp</code>	Gestor QProcess
<code>src/app/src/frontend_application.cpp</code>	Orquestador modo frontend
<code>src/app/src/backend_application.cpp</code>	Orquestador modo backend
<code>src/middleware/include/middleware/messages.h</code>	Constantes IPC
<code>src/middleware/src/client/internalclient.cpp</code>	Cliente IPC
<code>src/middleware/src/server/internalserver.cpp</code>	Servidor IPC
<code>docker/Dockerfile.dev</code>	Entorno Docker (sin cambios)
<code>.github/workflows/</code>	CI (puede necesitar actualización)
<code>README.md</code>	Instrucciones del proyecto
<code>docs/guias/</code>	Guías de desarrollo

Documento generado el 5 de julio de 2026 para el cambio *arquitectura-binario-unico*